



```
import { useEffect, useMemo, useState } from 'react';
import { FlatList, StatusBar, StyleSheet, TextInput, TouchableOpacity, View }
from 'react-native';
import { SafeAreaView } from 'react-native-safe-area-context';
```

```
import { ProductCard } from '@components/product-card';
import { ThemedText } from '@components/themed-text';
import { ThemedView } from '@components/themed-view';
import { BakeryColors } from '@constants/theme';
```

```
const PRODUCTS_URL = "http://localhost:3085/api/products";
```

```
type Locale = 'en' | 'th';
```

```
const translations = {
  en: {
    eyebrow: 'BAKERY SHOP',
    title: 'Bakery Stock',
    searchPlaceholder: 'Search pastries, categories...',
    filter: 'Filter ▼ ',
    addButton: 'Add New Pastry',
    sectionLabel: (count: number) => `All Products (${count})`,
    home: 'Home',
    add: 'Add',
    menu: 'Menu',
    settings: 'Settings',
  },
  th: {
    eyebrow: 'ร้านเบเกอรี่',
    title: 'สต็อกเบเกอรี่',
    searchPlaceholder: 'ค้นหาขนมและหมวดหมู่...',
    filter: 'ตัวกรอง ▼ ',
    addButton: 'เพิ่มขนมใหม่',
    sectionLabel: (count: number) => `สินค้าทั้งหมด (${count})`,
    home: 'หน้าแรก',
    add: 'เพิ่ม',
    menu: 'เมนู',
    settings: 'ตั้งค่า',
  },
};
```

```
export default function HomeScreen() {
  const [locale, setLocale] = useState<Locale>('en');
  const [rawProducts, setRawProducts] = useState<any[]>([]);

  const t = locale === 'en' ? translations.en : translations.th;
```

```

// Map ข้อมูลจาก MySQL ให้เข้ากับ Props ของ ProductCard
const visibleProducts = useMemo(() => {
  return rawProducts.map((item) => ({
    ...item,
    id: item.id?.toString(),
    // รองรับทั้งตารางแบบใหม่ (name, category) และแบบแยกภาษา (nameEn,
nameTh)
    name: locale === 'en'
      ? (item.nameEn || item.name)
      : (item.nameTh || item.name),
    category: locale === 'en'
      ? (item.categoryEn || item.category)
      : (item.categoryTh || item.category),
    // แปลง stock หรือ price สำหรับโชว์ในการ์ด
    price: item.price ? `${item.price} ฿` : (item.stock !== undefined ? `Stock:
${item.stock}` : ""),
    image: item.image || item.imageUrl,
  }));
}, [rawProducts, locale]);

useEffect(() => {
  async function loadProducts() {
    try {
      const response = await fetch(PRODUCTS_URL);
      const data = await response.json();
      setRawProducts(data);
    } catch (error) {
      console.error("Failed to load products:", error);
    }
  }

  void loadProducts();
}, []);

return (
  <SafeAreaView style={styles.container} edges={['top', 'bottom']}>
    <StatusBar barStyle="dark-content"
backgroundColor={BakeryColors.background} />

    <ThemedView style={styles.header}>
      <TouchableOpacity style={styles.categoryButton}>
        <ThemedText style={styles.categoryIcon}>☰</ThemedText>
      </TouchableOpacity>
      <View style={styles.titleWrap}>
        <ThemedText
style={styles.headerEyebrow}>{t.eyebrow}</ThemedText>
        <ThemedText style={styles.headerTitle}>{t.title}</ThemedText>

```

```

</View>
<View style={styles.headerActions}>
  <TouchableOpacity
    style={styles.langButton}
    onPress={() => setLocale((current) => (current === 'en' ? 'th' : 'en'))}
  >
    <ThemedText style={styles.langButtonText}>{locale === 'en' ? 'ไทย' :
'EN'}</ThemedText>
  </TouchableOpacity>
  <TouchableOpacity style={styles.profileButton}>
    <ThemedText style={styles.profileIcon}>👤 </ThemedText>
  </TouchableOpacity>
</View>
</ThemedView>

<View style={styles.searchContainer}>
  <View style={styles.searchBar}>
    <ThemedText style={styles.searchIcon}>🔍 </ThemedText>
    <TextInput
      style={styles.searchInput}
      placeholder={t.searchPlaceholder}
      placeholderTextColor={BakeryColors.textSecondary}
      editable={true}
    />
  </View>
  <TouchableOpacity style={styles.filterButton}>
    <ThemedText style={styles.filterIcon}>{t.filter}</ThemedText>
  </TouchableOpacity>
</View>

<View style={styles.addRow}>
  <TouchableOpacity style={styles.addButton}>
    <ThemedText style={styles.addButtonIcon}>+</ThemedText>
    <ThemedText
      style={styles.addButtonText}>{t.addButton}</ThemedText>
    </TouchableOpacity>
  </View>

<FlatList
  style={styles.list}
  contentContainerStyle={styles.shelfArea}
  data={visibleProducts}
  keyExtractor={({item}) => item.id}
  ListHeaderComponent={
    <ThemedText
      style={styles.sectionLabel}>{t.sectionLabel(visibleProducts.length)}</Themed
Text>
  }
  renderItem={({ item }) => <ProductCard product={item} />}
/>

```

```

<ThemedView style={styles.bottomNav}>
  <TouchableOpacity style={styles.navItem}>
    <ThemedText style={styles.navIcon}> 🏠 </ThemedText>
    <ThemedText style={styles.navText}>{t.home}</ThemedText>
  </TouchableOpacity>

  <TouchableOpacity style={styles.navItem}>
    <ThemedText style={styles.navIcon}> ➕ </ThemedText>
    <ThemedText style={styles.navText}>{t.add}</ThemedText>
  </TouchableOpacity>

  <TouchableOpacity style={styles.navItemActive}>
    <View style={styles.navActivePill}>
      <ThemedText style={styles.navIconActive}> 🍰 </ThemedText>
    </View>
    <ThemedText style={styles.navTextActive}>{t.menu}</ThemedText>
  </TouchableOpacity>

  <TouchableOpacity style={styles.navItem}>
    <ThemedText style={styles.navIcon}> ⚙️ </ThemedText>
    <ThemedText style={styles.navText}>{t.settings}</ThemedText>
  </TouchableOpacity>
</ThemedView>
</SafeAreaView>
);
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: BakeryColors.background,
  },
  header: {
    paddingVertical: 14,
    flexDirection: 'row',
    justifyContent: 'space-between',
    alignItems: 'center',
    paddingHorizontal: 20,
    backgroundColor: BakeryColors.surface,
    borderBottomWidth: 1,
    borderBottomColor: BakeryColors.border,
  },
  categoryButton: {
    width: 34,
    height: 34,
    justifyContent: 'center',
    alignItems: 'center',

```

```
},
categoryIcon: {
  fontSize: 18,
  color: BakeryColors.primaryDark,
},
titleWrap: {
  alignItems: 'center',
},
headerEyebrow: {
  fontSize: 11,
  color: BakeryColors.secondary,
  letterSpacing: 1,
  marginBottom: 2,
},
headerTitle: {
  fontSize: 20,
  fontWeight: '700',
  color: BakeryColors.textPrimary,
},
headerActions: {
  flexDirection: 'row',
  alignItems: 'center',
  gap: 8,
},
langButton: {
  paddingHorizontal: 10,
  paddingVertical: 6,
  borderRadius: 999,
  backgroundColor: BakeryColors.chip,
  borderWidth: 1,
  borderColor: BakeryColors.border,
},
langButtonText: {
  fontSize: 11,
  fontWeight: '700',
  color: BakeryColors.primaryDark,
},
profileButton: {
  width: 34,
  height: 34,
  backgroundColor: BakeryColors.primary,
  borderRadius: 17,
  justifyContent: 'center',
  alignItems: 'center',
```

```
},
profileIcon: {
  fontSize: 16,
  color: BakeryColors.surface,
},
searchContainer: {
  flexDirection: 'row',
  paddingHorizontal: 16,
  paddingTop: 16,
  paddingBottom: 10,
  gap: 12,
  alignItems: 'center',
},
searchBar: {
  flex: 1,
  flexDirection: 'row',
  backgroundColor: BakeryColors.surface,
  borderWidth: 1,
  borderColor: BakeryColors.border,
  borderRadius: 20,
  paddingHorizontal: 14,
  alignItems: 'center',
  height: 52,
},
searchIcon: {
  fontSize: 15,
  marginRight: 8,
  opacity: 0.7,
},
searchInput: {
  flex: 1,
  height: '100%',
  fontSize: 14,
  color: BakeryColors.textPrimary,
  paddingVertical: 0,
},
filterButton: {
  minWidth: 58,
  height: 52,
  paddingHorizontal: 14,
  backgroundColor: BakeryColors.surface,
  borderWidth: 1,
```

	<pre>borderColor: BakeryColors.border, borderRadius: 20, justifyContent: 'center', alignItems: 'center', }, filterIcon: { color: BakeryColors.primaryDark, fontSize: 18, fontWeight: '700', }, addRow: { paddingHorizontal: 16, paddingTop: 6, paddingBottom: 16, }, addButton: { backgroundColor: BakeryColors.primary, height: 48, borderRadius: 16, flexDirection: 'row', justifyContent: 'center', alignItems: 'center', gap: 6, shadowColor: BakeryColors.primaryDark, shadowOffset: { width: 0, height: 3 }, shadowOpacity: 0.2, shadowRadius: 6, elevation: 2, }, addButtonIcon: { color: BakeryColors.surface, fontSize: 16, fontWeight: '700', }, addButtonText: { color: BakeryColors.surface, fontWeight: '600', fontSize: 14, }, list: { flex: 1, }, shelfArea: {</pre>
--	---

```
paddingHorizontal: 20,
paddingTop: 4,
paddingBottom: 20,
},
sectionLabel: {
  fontSize: 13,
  fontWeight: '700',
  color: BakeryColors.secondary,
  marginBottom: 10,
},
bottomNav: {
  flexDirection: 'row',
  backgroundColor: BakeryColors.surface,
  borderTopWidth: 1,
  borderTopColor: BakeryColors.border,
  paddingVertical: 10,
  paddingBottom: 14,
},
navItem: {
  flex: 1,
  alignItems: 'center',
  justifyContent: 'center',
},
navItemActive: {
  flex: 1,
  alignItems: 'center',
  justifyContent: 'center',
},
navActivePill: {
  backgroundColor: BakeryColors.chip,
  borderRadius: 14,
  paddingHorizontal: 14,
  paddingVertical: 4,
  marginBottom: 4,
},
navIcon: {
  fontSize: 20,
  marginBottom: 4,
  opacity: 0.55,
},
navIconActive: {
  fontSize: 18,
},
},
```

	<pre>navText: { fontSize: 11, color: BakeryColors.textSecondary, }, navTextActive: { fontSize: 11, color: BakeryColors.primaryDark, fontWeight: '700', }, });</pre>
--	---